Muhammad Mahyar Zeb
Internee ID: NFS-2607-1125
Task6

Structured Outputs — Get Clean JSON From Any Prompt
Use Case: Customer Support Message Extraction

The goal is to extract structured information from customer support messages so the output can be directly used by an application.

JSON Schema

```
{
  "type": "object",
  "properties": {
    "name": {
      "type": "string"
    },
    "email": {
      "type": "string"
    },
    "issue_type": {
      "type": "string",
      "enum": ["billing", "technical", "account", "shipping", "other"]
    },
    "urgency": {
      "type": "string",
      "enum": ["low", "medium", "high"]
    }
  },
  "required": ["name", "email", "issue_type", "urgency"],
  "additionalProperties": false
}
```

Prompt

You are a customer support message classifier.
Read the customer's message and extract:
- name
- email
- issue_type
- urgency

Return ONLY valid JSON matching the provided schema. Do not include explanations, Markdown, code fences, or any text outside the JSON object.
Rules:

1. Always include all four fields.
2. If name or email is not provided, use an empty string.
3. issue_type must be one of: billing, technical, account, shipping, other.
4. urgency must be one of: low, medium, high.
5. Select the issue_type based on the customer's main problem.
6. Select urgency based on how time-sensitive or disruptive the problem is.

Testing With 5 Sample Inputs

Test 1:
Input: “Hi, I'm Sarah Khan (sarah@example.com). I was charged twice for my subscription this month. Please fix the duplicate charge.”
Output:
{"name":"Sarah Khan","email":"sarah@example.com","issue_type":"billing","urgency":"medium"}
Result: Valid JSON — PASS

Test 2:
Input: “My name is Ahmed and my email is ahmed@example.com. I can't log into my account even though I'm sure my password is correct. I need access urgently because I have a meeting in an hour.”
Output:
{"name":"Ahmed","email":"ahmed@example.com","issue_type":"account","urgency":"high"}
Result: Valid JSON — PASS

Test 3:
Input: “Hello, I'm Maria Lopez, maria@example.com. My package was supposed to arrive yesterday but the tracking page still says it hasn't shipped.”
Output:
{"name":"Maria Lopez","email":"maria@example.com","issue_type":"shipping","urgency":"medium"}
Result: Valid JSON — PASS

Test 4:
Input: “The app crashes every time I try to upload a photo. My name is James and you can reach me at james@example.com. It's annoying but I can still use the rest of the app.”
Output:
{"name":"James","email":"james@example.com","issue_type":"technical","urgency":"low"}
Result: Valid JSON — PASS

Test 5:
Input: “I have a question about whether your service supports international currencies. I don't have an account yet.”
Output:
{"name":"","email":"","issue_type":"other","urgency":"low"}
Result: Valid JSON — PASS

JSON Verification

The outputs can be verified using Python:

```
import json
outputs = [
  '{"name":"Sarah Khan","email":"sarah@example.com","issue_type":"billing","urgency":"medium"}',
  '{"name":"Ahmed","email":"ahmed@example.com","issue_type":"account","urgency":"high"}',
  '{"name":"Maria
Lopez","email":"maria@example.com","issue_type":"shipping","urgency":"medium"}',
  '{"name":"James","email":"james@example.com","issue_type":"technical","urgency":"low"}',
  '{"name":"","email":"","issue_type":"other","urgency":"low"}'
]

for i, output in enumerate(outputs, 1):
  try:
    data = json.loads(output)
    assert set(data.keys()) == {"name", "email", "issue_type", "urgency"}
    assert data["issue_type"] in {"billing", "technical", "account", "shipping", "other"}
    assert data["urgency"] in {"low", "medium", "high"}
    print(f"Test {i}: PASS")
  except (json.JSONDecodeError, AssertionError):
    print(f"Test {i}: FAIL")
```

Final Result: All 5 test cases produced valid JSON matching the defined schema. This shows how structured outputs can make AI responses predictable, machine-readable, and ready to use directly in an application.